51. Статические переменные

Ключевое слово static является одним из самых запутанных в языке C++. Оно имеет разные значения в разных ситуациях.

На уроке о **глобальных переменных** мы узнали, что, добавляя static к переменной, объявленной вне блока, мы определяем её как внутреннюю, то есть такую, которую можно использовать только в файле, в котором она определена.

Ключевое слово static можно применять и к переменным внутри блока, но тогда его значение будет другим. На уроке о **локальных переменных** мы узнали, что локальные переменные имеют автоматическую продолжительность жизни, т.е. создаются, когда блок начинается, и уничтожаются при выходе из него.

Использование **ключевого слова static** с локальными переменными изменяет их свойство продолжительности жизни с автоматического на статическое (или *«фиксированное»*). **Статическая переменная** (или ***«переменная со статической продолжительностью жизни»***) сохраняет свое значение даже после выхода из блока, в котором она определена. То есть она создается (и инициализируется) только один раз, а затем сохраняется на протяжении выполнения всей программы.

Рассмотрим разницу между переменными с автоматической и статической продолжительностями жизни.

Автоматическая продолжительность жизни (по умолчанию):

```
#include <iostream>

void incrementAndPrint()
{
    int value = 1; // автоматическая продолжительность жизни (по умолчанию)
    ++value;
    std::cout << value << std::endl;
} // переменная value уничтожается здесь

int main()
{
    incrementAndPrint();
    incrementAndPrint();
    incrementAndPrint();
}
```

Каждый раз, при вызове функции incrementAndPrint(), создается переменная `value`, которой присваивается значение `1`. Функция incrementAndPrint() увеличивает значение переменной до `2`, а затем выводит его. Когда incrementAndPrint() завершает свое выполнение, переменная выходит из области видимости и уничтожается.

Следовательно, результат выполнения программы:

```
2
2
2
```

Теперь рассмотрим статическую версию. Единственная разница между этими двумя программами только в добавлении ключевого слова static к переменной.

Статическая продолжительность жизни:

```
#include <iostream>

void incrementAndPrint()
{
    static int s_value = 1; // переменная s_value является статической
    ++s_value;
    std::cout << s_value << std::endl;
} // переменная s_value не уничтожается здесь, но становится недоступной

int main()
{
    incrementAndPrint();
    incrementAndPrint();
    incrementAndPrint();
}
```

Поскольку переменная `s_value` объявлена статической (с помощью ключевого слова static), то она создается и инициализируется только один раз. Кроме того, выходя из области видимости, она не уничтожается. Каждый раз, при вызове функции incrementAndPrint(), значение `s_value` увеличивается.

Результат выполнения программы:

```
2
3
4
```

Так же, как мы используем префикс `g_` с глобальными переменными, префикс `s_` принято использовать со статическими переменными. Обратите внимание, внутренние глобальные переменные (которые объявлены с использованием static) остаются с префиксом `g_`, а не с префиксом `s_`.

Зачем нужны статические локальные переменные? Одним из наиболее распространенных применений является генерация уникальных идентификаторов. При работе с большим количеством одинаковых объектов внутри программы часто бывает полезно присвоить каждому объекту отдельный уникальный идентификационный номер. Это легко осуществить, используя одну статическую локальную переменную:

```
int generateID()
{
    static int s_itemID = 0;
    return s_itemID++;
}
```

При первом вызове функции возвращается `0`. Во второй раз возвращается `1`. Затем `2` и каждый последующий вызов будет увеличивать эту переменную на единицу. Хороший способ генерации уникальных идентификаторов для похожих объектов? Хороший! Поскольку `s_itemID` — это локальная переменная, то она не может быть «изменена» другими функциями.

Статические переменные имеют некоторые преимущества глобальных переменных (они не уничтожаются до завершения программы), сохраняя при этом локальную область видимости. Таким образом, они намного безопаснее для использования, нежели глобальные переменные.

Тест

Какой эффект от добавления ключевого слова static к глобальной переменной? Какое влияние оно имеет на локальную переменную?

Ответ

Добавляя ключевое слово static к глобальной переменной, мы определяем её как внутреннюю, то есть такую, которую нельзя экспортировать и использовать в других файлах.

В случае с локальной переменной, добавление static определяет её как статическую, то есть она создается и инициализируется только один раз, и не уничтожается до самого конца программы.